



Banana Buddy — Documentação Final

*Fitness gamificado com prova social: um avatar-banana que reflete emocionalmente o esforço físico do usuário. **Gym Rats**, mas a banana apodrece se você sumir. Projeto acadêmico — Ibmec 2026.1. Atualizado em 2026-06-17.*

Este documento descreve o **que está de fato implementado** no protótipo (não a visão-alvo do `README.md`, que mistura roadmap 3D/AR). Para a memória técnica detalhada, ver `CLAUDE.md`; para o histórico de gaps, `GAPS.md`.

1. O Pitch

O problema

O mercado de fitness sofre com abandono (*churn*) após os primeiros 30 dias. Apps tradicionais focam em **dados frios** (gráficos, calorias) e falham em criar o laço emocional que sustenta disciplina.

A solução

O Banana Buddy humaniza o progresso com um **avatar-banana** cujo estado é espelho direto da sua atividade. O motor de retenção é o **loop de responsabilidade social do Gym Rats**, gamificado:

*Você registra um treino **com foto de prova** → seus colegas de clã veem no feed → o ranking atualiza → quem some tem a banana apodrecendo, e os amigos cutucam pra salvar.*

Dois pilares emocionais, ambos **reais no código** (não mockados):

- Pressão social** — Bananeiras (clãs), ranking, feed com foto, cutucada de resgate.
- A banana que apodrece** — decaimento visual progressivo por inatividade + ressurreição dramática.

2. Stack e Arquitetura (real)

Camada	Tecnologia
Front-end	Vite 6 + React 19 + TypeScript
UI	Tailwind CSS v4 + shadcn/ui (base-ui) + <code>motion</code> (animação) + <code>lucide-react</code>
Backend / dados	Supabase (Postgres + Auth + Storage), com RLS
Avatar	Animação 2D quadro-a-quadro (PNGs), 9 skins, 4 moods
Hospedagem	Vercel

- App single-file:** quase toda a lógica vive em `src/App.tsx` (telas, avatar, estado global como máquina de estados `useState<Screen>`).

- **Persistência:** Supabase, com `saveProfileNow` (debounce 500ms). Funções sociais em `lib/bananeiras.ts`.
 - **Migrations SQL** versionadas em `db/` (rodar no SQL Editor do Supabase).
-

3. Funcionalidades Implementadas

3.1 Avatar & Customização

- 9 skins (base, ballet, boxe, cycle, judo, run, soccer, swim, yoga); 4 moods (happy, on-fire, dead, zen).
- Desbloqueio por **conquista** (atingir meta do esporte) ou **compra** na loja com raios (moeda).
- Grid de customização com preview animado.

3.2 Autenticação

- Supabase Auth: cadastro/login por e-mail+senha (abas Entrar/Cadastrar).
- **Login com Google (OAuth)** real — `signInWithOAuth`. Criação automática de profile para contas novas.
- Trigger `handle_new_user` cria o profile no signup (`db/profiles.sql`).

3.3 Bananeiras (clãs sociais)

- Criar / entrar por **código** (RPCs `create_bananeira` / `join_bananeira`).
- **Ranking real** por soma de sessões (`bananeira_overview` , membros × profiles).
- **Mapa** com fundo pixel-art, posição por membro, pódio/coroa do líder, badge de fundador 🌱, esporte principal.
- **Cutucada de resgate:** membro em risco (2+ dias parado) aparece decaído; cutucar manda pop-up "Treinar agora"; se treinar em 24h, quem cutucou ganha +10 raios + badge 🍷 Apoiador. (`db/poke_rescue.sql`)
- Polling silencioso a cada 5s.

3.4 Registro de treino com prova (✅ entregue nesta fase)

Substituiu o antigo botão "+1". Fluxo de **check-in com foto**, estilo Gym Rats:

- **Foto de prova:** câmera **ou** galeria no celular; seletor no desktop. Preview + selo "✓ Válido".
- **GPS real:** `navigator.geolocation` (fallback mockado se negar).
- **Formulário:** Atividade, Título, Descrição, Duração, Distância, Calorias, Passos; hora automática.
- **Entradas unificadas:** tela de Conquistas e "+" do mapa abrem o **mesmo** `CheckInModal` (com slide-up animado).
- **Persistência real:** foto no **Supabase Storage** (bucket `checkins`) + linha na tabela `checkins` . (`db/checkins.sql`)

3.5 Feed social

- Botão 📷 no mapa → **feed da Bananeira** com check-ins de todos os membros (linhas compactas: miniatura + título + autor + horário).
- Tocar numa linha → **foto ampliada (lightbox)** com esporte, métricas e selo "Prova válida".
- Polling 5s. Funções `createCheckin` / `fetchBananeiraFeed` em `lib/bananeiras.ts`.

3.6 Streak (sequência)

- Contador de dias consecutivos no Dashboard e no card do mapa.
- **Escudo** a cada 7 dias (protege contra 1 falta); badges de título por marco (🏆 Persistente → Lendário).
- Modal de conquista ao bater 7 dias. (`db/streak.sql`)

3.7 Decaimento & Ressurreição

- 4 estágios de decaimento por dias sem treinar (via filtro CSS): saudável → amadurecendo 😊 → quase podre 🤢 → podre 💀 .
- Aplicado no mapa e no Dashboard, com texto de urgência.
- **Ressurreição**: treinar com banana podre dá +50 raios e modal dramático; co-membros são notificados. (`db/resurrection.sql`)

3.8 Economia & Metas

- **Raios** (moeda): +20 por check-in, +50 bônus de ressurreição.
- Metas pessoais que ativam o modo **On Fire** 🔥 .

4. Pontos Fracos Mapeados × Correções

Diagnóstico honesto feito em `GAPS.md` , comparando o protótipo com a tese "Gym Rats gamificado". Os dois pilares estavam **mockados**; foram tornados reais.

#	Ponto fraco diagnosticado	Status	O que foi feito
1	Loop social mockado (Bananeiras sem backend)	✅ Corrigido	Bananeiras reais no Supabase: criar/entrar por código, ranking real, cutucada de resgate com recompensa
2	Banana não apodrecia de verdade	✅ Corrigido	<code>bananaDecayState</code> com 4 estágios + ressurreição com bônus e notificação aos co-membros
3	Sem streak	✅ Corrigido	Streak persistido, escudo a cada 7 dias, badges de título, modal de conquista
4	Registro era só um botão "+1" (sem prova)	✅ Corrigido	Check-in com foto de prova , GPS real, métricas, persistência (Storage + <code>checkins</code>) e feed com lightbox
5	Integração de saúde / login social só fachada	🟡 Parcial	Login Google real implementado; integração de dados de saúde diagnosticada (ver §5)

Detalhe da correção do Gap 4 (foco desta fase)

O registro de treino era trivialmente "fraudável": clicava +1, ganhava 20 raios, sem data/duração/prova. Agora há **conceito real de sessão com prova social**:

- A foto é a legitimidade (o que torna o Gym Rats viciante).
- O check-in persiste e aparece no **feed do clã**, fechando o loop "registro → amigos veem → cobrança".
- Migration `db/checkins.sql` com RLS: leitura liberada a co-membros via helper `shares_bananeira` .

5. O que continua como fachada/roadmap (transparência)

- **Integração Apple Health**: inviável em web app — HealthKit é nativo iOS, sem API web. Exigiria app nativo (Capacitor/React Native).
- **Google Fit**: REST API descontinuada (migrando pro Health Connect, Android-nativo). Não vale o esforço.
- **Strava (caminho viável, adiado)**: OAuth + REST web-friendly; puxaria treino real (distância/duração/calorias) pra **auto-preencher o check-in**. Custo: ~meio dia + 1 função serverless (troca de token). Forte para o pitch se priorizado.

- **Login Apple:** removido — exige conta paga de Apple Developer (US\$ 99/ano).
 - **Avatar 3D/AR e geofencing real:** roadmap do `README.md` ; a animação 2D frame-a-frame comunica a ideia.
 - **Seeding de contas demo:** único pendente urgente — pré-popular uma Bananeira de demonstração pra o loop social e o feed ficarem demonstráveis ao vivo no pitch.
-

6. Como rodar

```
cd BananaBuddy/banana-buddy
npm install
npm run dev      # Vite na porta 3000
npm run build    # build de produção
```

Supabase: rodar as migrations de `db/` no SQL Editor. Variáveis `VITE_SUPABASE_URL` e `VITE_SUPABASE_ANON_KEY` no `.env` (e nas env vars da Vercel). Para o login Google, configurar o provider no Supabase + credenciais OAuth no Google Cloud.

Estrutura

```
src/App.tsx      - telas, avatar, estado global (single-file)
lib/bananeiras.ts - funções sociais (clãs, feed, check-in, ressurreição)
lib/supabase.ts  - client Supabase
db/*.sql         - migrations (profiles, bananeiras, streak, checkins, ...)
public/         - PNGs do avatar + imagens de referência
```